PRAKTIKUM CLOUD COMPUTING

DISASTER RECOVERY PLAN (DRP) AWAL

Sistem Flash Sale E-Commerce

Keterangan	Isi
Mata kuliah	Cloud Computing
Kegiatan	Tugas Terstruktur, Pertemuan 1: Planning & Application Setup
Kelompok	Kelompok 1
Anggota	Afrizal, Ardhiansyah, Luthfion, M. Diwan, Erridho
Kelas	H7D.7
Studi kasus	Sistem Flash Sale E-Commerce
Versi	1.0.2 (revisi: load balancer, waktu failover, bentrok data setelah failover); ditinjau ulang setelah Pertemuan 3 dan Pertemuan 4
Tanggal	[ISI]

1. Tujuan dan Ruang Lingkup

Dokumen ini menetapkan bagaimana sistem Flash Sale dipulihkan jika terjadi bencana, dengan fokus pada **kegagalan database utama (primary) saat transaksi puncak sedang berlangsung**. Sistem memakai dua server database, yaitu primary (master) dan replica (slave), sehingga matinya satu server tidak otomatis mematikan seluruh layanan. Dokumen ini juga membahas kegagalan load balancer dan risiko bentrok data jika primary lama hidup kembali setelah failover. Pemulihan harus menjamin tidak ada pesanan ganda dan tidak ada stok minus.

Dalam lingkup: container atau proses database primary dan replica, data stok dan order, layanan backend, load balancer Nginx, replikasi primary-replica, failover manual, fencing primary lama, serta proses backup dan restore. **Di luar lingkup:** kegagalan seluruh host, replikasi lintas lokasi, failover otomatis (Patroni atau repmgr), redundansi load balancer (dua node dengan virtual IP), dan serangan keamanan.

Catatan: load balancer ditambahkan pada Pertemuan 2 (docker-compose). Pada Pertemuan 1 aplikasi berjalan dengan satu instance backend tanpa load balancer, sehingga prosedur load balancer pada dokumen ini adalah rancangan yang diuji di Pertemuan 4.

2. Aset Kritis dan Dampak

Aset	Dampak jika hilang atau mati	Tingkat kritis
Data order	Pelanggan membayar tanpa bukti, terjadi sengketa	Sangat tinggi
Data stok	Overselling atau katalog salah	Sangat tinggi
Load balancer (Nginx)	Layanan tidak dapat diakses walau backend dan database sehat; tidak ada data hilang	Tinggi

Aset	Dampak jika hilang atau mati	Tingkat kritis
Database replica (read-only)	Salinan siap-promote hilang; pembacaan produk beralih ke primary	Sedang
Backend API	Penjualan berhenti total	Tinggi
Frontend	Pengguna tidak bisa berbelanja	Sedang
File backup	Tidak bisa restore jika volume hilang	Tinggi

3. Skenario Ancaman

ID	Ancaman	Kemungkinan	Dampak	Skor
T1	Container database primary mati atau terhapus saat checkout puncak	Sedang (2)	Tinggi (3)	**6**
T2	Volume data primary rusak atau hilang	Rendah (1)	Tinggi (3)	3
T3	Backend crash karena kelebihan beban	Tinggi (3)	Sedang (2)	**6**
T4	Kesalahan manusia (salah hapus atau ubah data)	Sedang (2)	Tinggi (3)	**6**
T5	File backup korup atau tidak bisa dipakai	Sedang (2)	Tinggi (3)	**6**
T6	Replica mati atau tertinggal jauh (jeda replikasi besar)	Sedang (2)	Rendah (1)	2
T7	Primary lama hidup kembali setelah replica dipromosikan (split-brain, data bentrok)	Sedang (2)	Tinggi (3)	**6**
T8	Load balancer mati	Rendah (1)	Tinggi (3)	3

Skor = kemungkinan × dampak (skala 1 sampai 3). **T1 adalah dasar DR Test kelompok ini.** T7 mendapat skor tertinggi pada ancaman baru karena kesalahannya sulit terlihat dan dapat merusak data. T8 berskor rendah karena Nginx sederhana dan tanpa state, tetapi dampaknya tinggi karena seluruh layanan terhenti.

4. Target RTO dan RPO

Metrik	Target	Penjelasan
RTO (Recovery Time Objective)	**≤ 5 menit**	Dihitung dari deteksi gagal (T0) sampai /health kembali ok dan data tervalidasi (T1)
RPO (Recovery Point Objective)	**≤ 1 menit**	Order yang boleh hilang setara maksimal 1 menit transaksi

4.1 Alasan Pemilihan Angka

- Flash sale berlangsung singkat (puluhan menit). Downtime lebih dari 5 menit menghilangkan sebagian besar peluang penjualan.
- Data order bernilai finansial, sehingga toleransi kehilangan data sangat kecil. Backup dijadwalkan setiap 1 menit, yaitu interval minimum cron.
- Replica menyediakan jalur pemulihan yang lebih cepat saat primary mati: pemulihan berupa promote replica, bukan membuat ulang container dan restore data.

4.2 RPO Menurut Jenis Kegagalan

Kegagalan	Data hilang?	RPO efektif
Hanya container DB dihapus, Docker Volume utuh	Tidak ada data yang sudah commit hilang. Hanya transaksi yang sedang berjalan saat crash yang tidak selesai.	≈ 0
Volume ikut hilang, restore dari dump	Order sejak dump terakhir hilang	≤ interval backup (1 menit)
Primary mati, replica sehat dan dipromosikan (Jalur Failover)	Order yang sudah commit di primary tetapi belum sampai ke replica (replikasi asinkron). Order ini tetap tersimpan di primary lama sehingga dapat diselamatkan lewat rekonsiliasi (bagian 7.4)	Sebesar jeda replikasi (kecil, diukur di Pertemuan 4)
Load balancer mati	Tidak ada. Load balancer tidak menyimpan data; request yang sedang berjalan diulang klien dengan key yang sama	0

4.3 Perkiraan Anggaran Waktu RTO

Angka berikut baru perkiraan dan akan diukur sebenarnya di Pertemuan 4. Tabel pertama untuk Jalur Recreate (container primary dibuat ulang), tabel kedua untuk Jalur Failover (replica dipromosikan).

Jalur Recreate (Skenario T1 dan T2):

Tahap	Perkiraan
Deteksi (health check gagal)	0,5 menit
Diagnosis (container saja atau volume juga hilang)	1 menit
Membuat ulang container database	0,5 menit
Restore data (bila perlu)	1 menit
Validasi integritas	1 menit
Menyambungkan ulang backend	0,5 menit
Total	**± 4,5 menit**

Jalur Failover (Skenario T1 dengan replica sehat):

Tahap	Perkiraan
Deteksi (health check gagal)	0,5 menit
Diagnosis (/health: primary down, replica up, jeda replikasi kecil) dan fencing primary lama	0,75 menit
Promosikan replica (pg_ctl promote; prosesnya sendiri beberapa detik)	0,25 menit
Ubah konfigurasi backend dan jalankan ulang	0,5 menit
Validasi integritas	1 menit
Total	**± 3 menit**

Anggaran ini ketat, sehingga pemulihan sebaiknya dijalankan dengan skrip, bukan perintah manual satu per satu. Jalur Failover diperkirakan lebih cepat karena tidak perlu membuat

ulang container maupun restore data, tetapi bergantung pada replica yang sehat. Waktu ini mengasumsikan operator siaga selama flash sale. Sebagai pembanding, failover otomatis (Patroni atau repmgr) umumnya selesai dalam puluhan detik, tetapi di luar lingkup praktikum.

4.4 Ringkasan Perkiraan Waktu Pemulihan per Jenis Kegagalan

Kegagalan	Jalur pemulihan	Perkiraan waktu	Data hilang
Load balancer mati	Restart policy otomatis; manual: docker compose up -d lb	Detik (otomatis); ≤ 1 menit (manual)	Tidak ada
Satu instance backend mati	Load balancer mengalihkan ke instance lain	Tanpa downtime layanan; request yang sedang berjalan diulang klien	Tidak ada
Primary mati, replica sehat	Jalur Failover	± 3 menit	Sebesar jeda replikasi (dapat direkonsiliasi)
Primary mati, replica tidak sehat	Jalur Recreate (volume utuh)	± 4,5 menit	≈ 0
Volume primary hilang	Failover, atau restore dari dump	± 3 menit (failover); ± 4,5 menit (restore)	0 sampai ≤ 1 menit
Primary lama hidup kembali	Fencing, rekonsiliasi, bangun ulang jadi replica	Tidak memengaruhi layanan; dikerjakan setelah stabil	Tidak ada jika prosedur 7.4 diikuti

4.5 Dampak bagi Pengguna Selama Failover

Endpoint	Selama failover (± 3 menit)	Setelah pulih
GET /products	Tetap 200, dibaca dari replica (X-Read-From: replica)	Normal
POST /checkout	503; klien mengulang dengan Idempotency-Key yang sama	Normal, ke primary baru
GET /orders/:id dan /stats	503 (dibaca dari primary)	Normal, ke primary baru
GET /health	503 karena primary down; field replica tetap dilaporkan	200

5. Strategi Backup

Lapis	Mekanisme	Frekuensi	Melindungi dari
1	Docker Named Volume untuk data PostgreSQL primary dan replica (dibangun di Pertemuan 2)	Kontinu	T1 (container hilang)
2	Replica PostgreSQL melalui streaming replication asinkron (primary-replica, Pertemuan 1)	Kontinu	T1 (replica dapat dipromosikan saat primary mati)
3	pg_dump format custom (terkompresi) ke folder backups (Pertemuan 4)	Setiap 1 menit	T2, T4
4	Retensi 10 file terakhir, file lama dihapus otomatis	Setiap backup	Disk penuh, T5
5	Uji restore ke database sementara	Sekali sebelum DR Test	T5

Mengapa pg_dump: menghasilkan snapshot konsisten tanpa mengunci tabel dari penulisan (berbasis MVCC), sehingga tidak membebani transaksi flash sale. Stok dan order dalam satu dump berasal dari satu titik waktu, sehingga invarian stok tetap terjaga.

Mengapa replica tidak menggantikan pg_dump: perintah destruktif seperti DROP TABLE atau DELETE tanpa WHERE di primary ikut tereplikasi ke replica dalam hitungan detik. Replica melindungi dari matinya server (T1), sedangkan pg_dump melindungi dari data yang rusak atau terhapus (T2, T4, T5).

Aturan operasional:

1. File ditulis ke nama sementara lalu diganti namanya setelah selesai, agar file setengah jadi tidak dianggap valid.
2. Backup disimpan terpisah dari volume data.
3. Setiap backup dicatat ke log, dan backup yang gagal harus terlihat.

Contoh perintah (draf, difinalisasi di Pertemuan 4):

```
# Backup
pg_dump -U postgres -d flashsale -Fc -f backups/flashsale_$(date +%Y%m%d_%H%M%S).dump

# Restore
pg_restore -U postgres -d flashsale --clean --if-exists backups/<nama_file>.dump
```

6. Deteksi Insiden

Sinyal	Sumber	Tindakan
/health mengembalikan 503 dua kali berturut-turut	Pemantauan manual atau skrip	Nyatakan insiden dan catat waktu T0. Periksa field replica pada /health untuk memilih jalur pemulihan (bagian 7)
/health menampilkan replica "down" atau replica_lag_seconds terus naik	Pemantauan manual atau skrip	Perbaiki atau buat ulang replica (bagian 7.3); jangan mengandalkan failover sebelum replica sehat
Request ke port load balancer gagal (connection refused atau timeout), tetapi backend yang diakses langsung menjawab 200	Pemantauan manual atau skrip	Load balancer mati: ikuti bagian 7.5
Container primary lama terlihat berjalan lagi setelah failover	docker ps atau pemantauan manual	Risiko split-brain: ikuti bagian 7.4 dan jangan arahkan backend ke sana
Error rate checkout melonjak	Keluaran load test	Periksa database

7. Prosedur Pemulihan

Nama container pada perintah di bawah (pg-primary, pg-replica, lb) adalah contoh dan menyesuaikan konfigurasi docker-compose di Pertemuan 2.

7.1 Skenario T1: Container Database Primary Mati atau Terhapus, Volume Utuh

Ini adalah skenario DR Test kelompok. Tersedia dua jalur pemulihan; pilih berdasarkan status replica pada /health.

Jalur Failover (replica sehat dan jeda replikasi kecil; diprioritaskan karena paling cepat dan tidak bergantung pada volume primary):

Langkah	Aksi
1	Catat T0 dan umumkan insiden
2	Buka /health: db = down dan replica = up. Catat replica_lag_seconds terakhir sebagai perkiraan data yang belum tereplikasi
3	**Fencing:** pastikan primary lama berhenti dan tidak restart otomatis: docker stop pg-primary lalu docker update --restart=no pg-primary. Jangan menjalankan docker compose up -d untuk service primary lama
4	Promosikan replica: ./scripts/promote-replica.sh (pg_ctl promote). Pastikan SELECT pg_is_in_recovery() bernilai false
5	Ubah .env backend: DB_PORT ke port replica (5433) dan kosongkan DB_REPLICA_HOST, lalu jalankan ulang backend (semua instance)
6	Jalankan validasi integritas (bagian 9)
7	Pastikan /health kembali ok, catat T1, dan hitung RTO = T1 − T0
8	Klien mengulang request yang gagal dengan key yang sama
9	Setelah layanan stabil, selamatkan order ekor dari primary lama (bagian 7.4), lalu bangun replica baru dari primary yang baru dengan scripts/setup-replica.sh

Jalur Recreate (replica juga tidak sehat; container primary dibuat ulang dengan volume yang sama):

Langkah	Aksi
1	Catat T0 dan umumkan insiden
2	Pastikan volume data masih ada
3	Buat ulang container database dengan volume yang sama
4	Tunggu database siap menerima koneksi
5	Jalankan validasi integritas (bagian 9)
6	Pastikan /health kembali ok dan backend tersambung
7	Catat T1 dan hitung RTO = T1 − T0
8	Klien mengulang request yang gagal dengan key yang sama

7.2 Skenario T2: Volume Hilang atau Rusak

Jika replica masih sehat, Jalur Failover pada bagian 7.1 juga dapat dipakai karena tidak membutuhkan volume primary yang hilang. Restore dari dump dipakai bila replica juga rusak atau bila penyebabnya adalah data yang salah dihapus (kesalahan ikut tereplikasi ke replica).

Langkah	Aksi
1	Catat T0, hentikan backend agar tidak menulis ke database yang belum pulih
2	Buat container dan volume database baru, lalu buat database flashsale
3	Pilih dump terbaru yang valid
4	Restore dari dump

Langkah	Aksi
5	Validasi integritas (bagian 9)
6	Jalankan ulang backend, cek /health, catat T1
7	Laporkan jumlah order yang hilang (selisih waktu dump terakhir dan crash) sebagai RPO aktual
8	Bangun ulang replica dari primary hasil restore dengan scripts/setup-replica.sh

7.3 Skenario Lain

- **T3, backend crash:** restart backend. Pool koneksi dibuat ulang otomatis dan tidak ada data yang hilang. Dengan load balancer, instance backend yang lain tetap melayani selama satu instance dipulihkan.
- **T4, kesalahan manusia:** hentikan penulisan, pilih dump terakhir sebelum kesalahan, lalu restore. Replica tidak membantu karena kesalahan ikut tereplikasi.
- **T5, dump korup:** coba dump sebelumnya. Itulah alasan retensi 10 file.
- **T6, replica mati atau jeda besar:** backend otomatis membaca dari primary (X-Read-From: primary) sehingga checkout tidak terpengaruh. Buat ulang replica dengan scripts/setup-replica.sh. Selama replica tidak ada, tidak ada kandidat failover cepat, sehingga perbaikannya diprioritaskan.

7.4 Skenario T7: Primary Lama Hidup Kembali Setelah Failover (Bentrok Data)

Latar belakang. Setelah replica dipromosikan, ia membuka timeline baru dan menerima penulisan. Primary lama tidak tahu hal itu. Jika ia hidup kembali dan menerima penulisan, ada dua server yang merasa primary dan datanya berbeda (split-brain). PostgreSQL tidak dapat menggabungkan keduanya otomatis. Primary baru tetap konsisten, tetapi order yang hanya ada di primary lama (order ekor) tidak ada di primary baru.

Dua kondisi:

Kondisi	Tindakan
Primary lama hidup kembali **sebelum** replica dipromosikan (misalnya hanya restart singkat)	Tidak perlu failover. Replica tersambung lagi dan melanjutkan streaming. Cek /health sampai replica up dan jeda replikasi kecil
Primary lama hidup kembali **setelah** replica dipromosikan	Jalankan langkah di bawah. Jangan arahkan backend ke primary lama

Langkah	Aksi
1	Pastikan backend hanya menulis ke primary baru (.env). Hentikan primary lama bila masih berjalan di port yang dikenal backend
2	Jalankan data primary lama secara terisolasi di port lain (contoh di bawah) untuk dibaca, bukan untuk menerima penulisan dari aplikasi
3	Bandingkan timeline_id kedua server. Nilai berbeda menandakan divergensi
4	Cari idempotency_key yang hanya ada di primary lama (order ekor)
5	Putar ulang order ekor ke primary baru lewat POST /checkout dengan key yang sama. 201 = diselamatkan, 200 = ternyata sudah ada, 409 = stok sudah habis (kompensasi: refund atau pemberitahuan ke pembeli)

Langkah	Aksi
6	Catat hasil rekonsiliasi, hapus container primary lama, lalu bangun ulang sebagai replica dari primary baru (scripts/setup-replica.sh)
7	Jalankan validasi integritas (bagian 9) pada primary baru dan pastikan replica baru berada dalam mode recovery

Perintah contoh (sesuaikan nama volume, port, dan kredensial):

```
# Langkah 2: jalankan data primary lama terisolasi di port 5434
docker run -d --name pg-old-primary -p 5434:5432 \
  -e POSTGRES_PASSWORD=postgres \
  -v <volume-primary-lama>:/var/lib/postgresql/data postgres:16

# Langkah 3: bandingkan timeline (5434 = primary lama, 5433 = primary baru)
psql -h localhost -p 5434 -U postgres -d flashsale -Atc \
  "SELECT timeline_id FROM pg_control_checkpoint();"
psql -h localhost -p 5433 -U postgres -d flashsale -Atc \
  "SELECT timeline_id FROM pg_control_checkpoint();"

# Langkah 4: order yang hanya ada di primary lama
psql -h localhost -p 5434 -U postgres -d flashsale -Atc \
  "SELECT idempotency_key FROM orders" | LC_ALL=C sort > old_keys.txt
psql -h localhost -p 5433 -U postgres -d flashsale -Atc \
  "SELECT idempotency_key FROM orders" | LC_ALL=C sort > new_keys.txt
LC_ALL=C comm -23 old_keys.txt new_keys.txt > order_ekor.txt

# Langkah 5: putar ulang order ekor ke primary baru lewat API
psql -h localhost -p 5434 -U postgres -d flashsale -Atc \
  "SELECT idempotency_key, user_id, product_id, qty FROM orders" \
  | LC_ALL=C sort -t"|" -k1,1 > old_full.txt
LC_ALL=C join -t"|" order_ekor.txt old_full.txt > replay.txt
while IFS="|" read -r key uid pid qty; do
  curl -s -o /dev/null -w "$key %{http_code}\n" -X POST http://localhost:8080/checkout \
    -H "Content-Type: application/json" -H "Idempotency-Key: $key" \
    -d "{\"user_id\":$uid,\"product_id\":$pid,\"qty\":$qty}"
done < replay.txt

# Langkah 6: buang primary lama, bangun ulang sebagai replica
docker rm -f pg-old-primary
./scripts/setup-replica.sh   # arahkan sumbernya ke primary baru
```

Karena idempotency_key bersifat UNIQUE, memutar ulang order yang ternyata sudah ada tidak membuat order ganda. Alternatif pg_rewind dapat dipakai untuk menyambungkan primary lama sebagai replica tanpa salin penuh, tetapi memerlukan wal_log_hints atau data checksums aktif; praktikum memakai pg_basebackup yang lebih sederhana. Replikasi sinkron akan menghilangkan order ekor (RPO = 0) namun menambah latensi checkout dan menghentikan penulisan bila replica mati, sehingga tidak dipilih di sini.

7.5 Skenario T8: Load Balancer Mati

Load balancer tidak menyimpan state, sehingga tidak ada data yang hilang. Yang terdampak adalah akses pengguna.

Langkah	Aksi
1	Catat T0. Konfirmasi: request ke port load balancer gagal, sedangkan backend yang diakses langsung menjawab 200
2	Periksa status container: docker ps -a. Bila restart policy unless-stopped aktif dan prosesnya crash, Docker biasanya sudah menjalankannya kembali dalam hitungan detik
3	Bila container terhapus atau dihentikan manual: docker compose up -d lb

Langkah	Aksi
4	Bila gagal start, periksa konfigurasi: docker logs lb dan nginx -t di dalam container, perbaiki, lalu jalankan ulang
5	Pastikan request lewat load balancer kembali berhasil (curl http://localhost:8080/health). Catat T1 dan hitung waktu pulih
6	Klien mengulang request yang gagal dengan key yang sama. Hasilnya 201 atau 200 tanpa order ganda
7	Jalankan /stats untuk memastikan invarian tetap benar

Satu instance backend mati: Nginx menandai instance itu tidak sehat (max_fails dan fail_timeout) dan mengirim trafik ke instance lain, sehingga tidak ada downtime layanan. Nginx tidak mengulang otomatis request POST yang sudah terkirim ke backend; pengulangan dilakukan klien dengan key yang sama. **Jalur darurat:** selama load balancer pulih, backend dapat diakses langsung lewat portnya secara manual. **Produksi:** dua node load balancer dengan virtual IP (keepalived) atau load balancer terkelola penyedia cloud, di luar lingkup praktikum.

8. Rencana DR Test (Dieksekusi di Pertemuan 4)

Skenario resmi kelompok: hapus paksa container database primary tepat saat stress test checkout flash sale berjalan, pulihkan (dengan failover ke replica atau dari backup terbaru), dan buktikan tidak ada pesanan ganda maupun stok minus.

#	Tahap	Aksi	Bukti
1	Baseline	Reset data, catat /stats awal	Screenshot
2	Beban	Jalankan load test checkout	Terminal load test
3	Bencana	Saat beban berjalan, hapus paksa container DB primary; catat T0	Screenshot error 503
4	Pemulihan	Jalankan prosedur bagian 7 (Uji A: Jalur Recreate, Uji B: restore dump, Uji C: Jalur Failover)	Log perintah dan waktu
5	Layanan pulih	/health kembali ok; catat T1	Screenshot
6	Retry klien	Ulangi request gagal dengan key yang sama	Respons 200 atau 201
7	Validasi	Jalankan query bagian 9 dan /stats	Screenshot hasil
8	Hasil	Hitung RTO dan RPO aktual	Tabel hasil

Lima variasi uji agar laporan kuat:

- **Uji A** menghapus container saja (membuktikan fungsi Docker Volume).
- **Uji B** menghapus container sekaligus volume lalu restore dari dump (membuktikan backup dan mengukur RPO nyata).
- **Uji C** mematikan container primary lalu failover ke replica dengan Jalur Failover (membuktikan arsitektur master-slave, mengukur RTO jalur failover dan RPO akibat jeda replikasi).
- **Uji D** menghentikan container load balancer saat beban berjalan (mengukur waktu pulih load balancer dan membuktikan tidak ada data hilang atau order ganda).

- **Uji E** menghidupkan kembali primary lama setelah Uji C secara terisolasi, lalu menjalankan prosedur 7.4 (membuktikan divergensi terdeteksi, order ekor direkonsiliasi, dan primary lama menjadi replica).

9. Kriteria Keberhasilan dan Validasi Integritas

DR Test dinyatakan berhasil jika:

- RTO aktual ≤ 5 menit dan RPO aktual ≤ 1 menit
- Tidak ada stok minus
- Tidak ada pesanan ganda
- Total terjual sama dengan stok yang berkurang
- Pada Uji C, replica berhasil dipromosikan (pg_is_in_recovery() = false) dan backend menulis ke primary yang baru
- Pada Uji D, request lewat load balancer kembali berhasil ≤ 1 menit dan tidak ada data hilang
- Pada Uji E, timeline_id berbeda terdeteksi, order ekor direkonsiliasi tanpa order ganda, dan replica baru berada dalam mode recovery (pg_is_in_recovery() = true)

Query validasi:

```
-- V1. Stok minus -> harus 0 baris
SELECT id, name, stock FROM products WHERE stock < 0;

-- V2. Order ganda -> harus 0 baris
SELECT idempotency_key, COUNT(*) FROM orders
GROUP BY idempotency_key HAVING COUNT(*) > 1;

-- V3. Invarian stok vs order -> kolom selisih harus 0 untuk semua produk
SELECT p.id, p.name, p.initial_stock, p.stock,
       p.initial_stock - p.stock AS terpakai,
       COALESCE(SUM(o.qty), 0)   AS terjual,
       (p.initial_stock - p.stock) - COALESCE(SUM(o.qty), 0) AS selisih
FROM products p LEFT JOIN orders o ON o.product_id = p.id
GROUP BY p.id ORDER BY p.id;

-- V4. Server yang dilayani backend harus primary yang bisa ditulisi -> harus false
SELECT pg_is_in_recovery();
```

Selain query di atas, endpoint /stats harus menampilkan "consistent": true. Setelah failover, semua query dijalankan pada server yang menjadi primary baru.

Template Tabel Hasil Uji A sampai C (diisi di Pertemuan 4)

Metrik	Target	Uji A	Uji B	Uji C	Lulus?
Jalur pemulihan	-	Recreate	Restore dump	Failover	
T0 (waktu bencana)	-	[ISI]	[ISI]	[ISI]	
T1 (sistem pulih)	-	[ISI]	[ISI]	[ISI]	
RTO aktual	≤ 5 menit	[ISI]	[ISI]	[ISI]	☐
Order hilang (RPO aktual)	≤ 1 menit	[ISI]	[ISI]	[ISI]	☐

Metrik	Target	Uji A	Uji B	Uji C	Lulus?
Stok minus	0	[ISI]	[ISI]	[ISI]	☐
Pesanan ganda	0	[ISI]	[ISI]	[ISI]	☐
Invarian stok = terjual	Semua produk	[ISI]	[ISI]	[ISI]	☐

Template Tabel Hasil Uji D dan E (diisi di Pertemuan 4)

Metrik	Target	Hasil	Lulus?
Uji D: waktu pulih load balancer (T1 − T0)	≤ 1 menit	[ISI]	☐
Uji D: jumlah request gagal selama load balancer mati	Dicatat	[ISI]	
Uji D: order hilang atau ganda setelah pulih	0	[ISI]	☐
Uji E: timeline_id primary lama dan primary baru	Berbeda	[ISI]	☐
Uji E: jumlah order ekor di primary lama	Dicatat	[ISI]	
Uji E: hasil putar ulang (201 / 200 / 409)	Tanpa order ganda	[ISI]	☐
Uji E: stok minus dan invarian di primary baru	0 dan sesuai	[ISI]	☐
Uji E: primary lama menjadi replica (pg_is_in_recovery() = true)	true	[ISI]	☐

10. Keterbatasan yang Diakui

1. Failover bersifat manual. Arsitektur primary-replica menghilangkan single point of failure pada data (replica menyimpan salinan), tetapi pemulihan tetap dijalankan operator berdasarkan prosedur. Failover otomatis memerlukan Patroni atau repmgr dan di luar lingkup praktikum.
2. Replikasi bersifat asinkron: order yang baru commit di primary dan belum sampai ke replica saat primary mati dapat hilang dari primary baru (RPO lebih dari 0, sebesar jeda replikasi). Order itu masih ada di primary lama dan dapat diselamatkan lewat rekonsiliasi (7.4). Retry dengan idempotency key hanya menolong klien yang belum menerima respons sukses.
3. Fencing primary lama dilakukan manual. Tidak ada mekanisme otomatis yang mencegah primary lama dihidupkan kembali dan ditulisi oleh operator yang tidak mengikuti prosedur.
4. Load balancer hanya satu node. Pada kegagalannya layanan terhenti sampai load balancer dipulihkan. Redundansi dua node dengan virtual IP di luar lingkup praktikum.
5. Pada kegagalan volume, RPO lebih dari 0: order sejak dump terakhir bisa hilang. Mitigasinya adalah retry berbasis idempotency key.
6. Backup dan replica berada di host yang sama, sehingga jika seluruh host hilang keduanya ikut hilang.
7. Seluruh angka RTO pada dokumen ini masih perkiraan sampai diukur di Pertemuan 4.

Riwayat Revisi

Versi	Perubahan
1.0	Versi awal (Pertemuan 1)
1.0.1	Revisi setelah masukan dosen: arsitektur primary-replica (master-slave), Jalur Failover, ancaman T6, dan Uji C
1.0.2	Revisi kedua setelah masukan dosen: load balancer mati (T8, 7.5, Uji D), lama proses failover (4.3 sampai 4.5), bentrok data setelah failover (T7, 7.4, Uji E), dan tanggapan atas masukan dosen
1.1	Penyesuaian setelah stress test (Pertemuan 3)
2.0	Final setelah DR Test (Pertemuan 4)